Link prediction performance with different node embedding methods in a heterogeneous disease-gene-drug network

Lorenzo Berlese Minseok Kim Matteo Meneghin

January 16, 2026

1 Abstract

The objective of this work is to test some node embedding methods and then make use of them in a downstream machine learning task. The task is edge prediction in a heterogeneous graph, for which it will be assessed accuracy of prediction in a scenario of missing edges, removed from the entire graph either in the training session and the test part of the experiment. The heterogeneous graph is a biological one. The node types are Gene, Disease and Drug, while the edges represent for example the correlation between a gene and a disease, the interaction between proteins, the comorbidity of two pathologies, the positive action of a chemical compound on the outcome of a disease, etc.

The project code is available at [1].

2 Data preprocessing

A complete description of the data processing and the code used can be found at [5].

2.1 Datasets considered

Data are taken from the following datasets:
- **VitaGraph** [4] available at `https://www.kaggle.com/datasets/gianlucadecarlods/vitagraph`.
- Comparative Toxicogenomics Database (**ctd**) offers different datasets at `https://ctdbase.org/downloads`, we used 5 of them.
- Stanford Biomedical Network Dataset Collection (**SNAP**) offers different datasets at `https://snap.stanford.edu/biodata/index.html`, we used 8 of them.

2.2 How datasets have been merged

The problem is that nodes in the different files have different identifiers, so first we counted them to understand which was more relevant.
After that we chose to keep as primary: Gene::NCBI, Disease::MESH, Drug::Pubchem_Compound. For other relevant types, we found a way to map them to a 'primary' type.

Disease: from DOID/OMIM to MESH For converting from DOID and OMIM to MESH we used two different files as 'map' to try to cover more DOID and OMIM possible.

Gene: from Uniprot to NCBI For converting the ones from Uniprot we used `mapUniProt()` from `UniProt.ws` (a R library) that uses Web Services of the official Uniprot site.

Drug: from Drugbank/MESH to Pubchem_Compound For retrieving Pubchem_Compound we used `get_cid()` from `webchem` (a R library) that uses the API of the official site of Pubchem. The same function allows us to query Drugbank and MESH terms.

Our final dataset [6] has 23 334 006 edges and the statistics of nodes can be seen in Table 1.

3 General Pipeline

The pipeline is run by `run_pipeline.py` that orchestrates the entire link prediction workflow using k-fold cross-validation.

Type	Number of nodes
Gene	56571
Disease	12766
Drug	15232
Total	104451

Table 1: Nodes in our final dataset

3.1 Pipeline Overview

The pipeline follows this workflow:
First, the `load_graph_data()` function in `data_loader.py` loads edge lists and detects whether the graph is heterogeneous (based on node type prefixes like Gene::, Disease::, Drug::).
Then `create_cv_folds()` partitions edges into k folds. For each fold, `prepare_fold_data()` creates training graph with test edges removed, positive and negative training samples, positive and negative test samples.
For each embedding method (in embeddings/ directory) we generates node representations using only the training graph, and then `run_single_fold()` trains a logistic regression classifier and evaluates predictions.

3.2 Negative Edges

The `generate_negative_samples()` function in `data_loader.py` creates non-existent edges by randomly pairing nodes that are not connected in the training graph. The negative sampling process is critical for creating a realistic link prediction scenario. The implementation generates negative samples by first randomly selecting pairs of nodes from the training graph and then verifying that no edge exists between the selected nodes and ensuring the same node is not paired with itself. We maintain a 1:1 ratio of negative to positive samples (configurable).

This approach simulates the real-world scenario, in which we must distinguish between true biological interactions and random node pairs that do not interact.

4 Embeddings

We tried some shallow embeddings, with some differences between them. We can divide the shallow methods tested into two subsets: those that follow a random walk strategy (DeepWalk, node2vec and metapath2vec) and graphwave, which instead takes advantage of "heat wavelet diffusion patterns". In addition, we tried some deep embeddings such as GATher, graphSAGE and another implementation of a GCN. For the latter methods, we decided to embed the nodes of the graph in a 16-bit space to accommodate computability constraints on time.

4.1 DeepWalk

We implemented the DeepWalk [7] algorithm using the gensim Python library to access a Word2vec model to generate embedding starting from random walks. We used the skip-gram version of embedding generation instead of the CBOW version. The skip-gram model is the one we studied in class.

Moreover, in the gensim implementation of the skip-gram there is negative sampling.

The main methods of the created DeepWalkEmbedder class are:

- `_generate_random_walks`: takes a NetworkX graph and performs `num_walks` walks of length `walk_length` on each node of the graph. It returns a list of lists of IDs (integers in the homogeneous dataset used until now, but they will be an alphanumerical IDs in the final dataset)
- `generate_embeddings`: creates the walks through `_generate_random_walks`, uses the Word2Vec model upon them to create the embeddings. It returns a dictionary of node:embedding elements

4.2 node2vec

We proceeded implementing the node2vec algorithm applying the biased random walks idea of the original paper [3], to then use the background probabilities obtained to train a Word2vec model as before.

To fix the right value for the parameters p and q of the biased random walk, we tried the following values of p and q, using different runs of the algorithm changing the parameters (in order to parallelize the work on th DEI server computers). We did not manage to use some suggestions from the original paper, since the values they used have been omitted (and in general the paper suggested to find the optimal ones in the specific context the algorithm is used).

The main methods of the Node2VecEmbedder class are:

- `_biased_choice`: calculates the probabilities for a successive step of the walk using the parameters p and q.

- `_node2vec_walk`: performs one random walk of length `walk_length` starting from a node
- `_generate_walks`: performs `num_walks` walks for each node in the graph using `_node2vec_walk` method
- `generate_embeddings`: creates the walks through `_generate_walks`, uses the Word2Vec model upon them to create the embeddings. Returns a dictionary of node:embedding elements.

4.3 metapath2vec

We then tried metapath2vec, an algorithm suited for heterogeneous graph. The novelty of it is that it is possible to fix a "metapath", which is a sequence of node types, that a random walk has to respect. In this way, if we think of nodes as words and paths as sentences, with a metapath we calibrate a different type of "semantic" in the sentence construction.

An example of a metapath is: ["Gene", "Disease", "Gene", "Gene", "Drug", "Gene"]. On top of this, the code is written to specify a set of metapaths upon which random walks are performed, to generate sentences with a set of different semantics to be analysed by Word2vec.

This feature permits to grasp different features of the neighbourhood off a node, creating more rich embeddings.

The path performed is not biased as in node2vec, but it gives the same weight to every member of the neighbourhood.

4.4 graphwave

This method is described in [2].

This type of embedding is different from the previous one because, in order to understand the topological structure of the neighbourhood of a node, it does not make use of random walks, but "heat wavelet diffusion patterns". At a high level, it is like spreading from a node a unit of heat and seeing how the energy distributes in the near nodes. On a more specific level, you take the Laplacian of the graph (which represents how the graph will spread the heat), you create a heat kernel with it (which is the function of the heat transmission from a generic node for all the other nodes), and then you multiply this kernel for the one-hot encoding of the node from which you want to apply the energy unit.

This last mathematical object is the wavelet.

The result of this calculation is a vector of size n (with n = number of nodes in the graph), where each entry is the amount of heat that reached a node. It is possible to apply the heat kernel to the output of the previous calculation to see the diffusion of heat at the following time step.

You stop after τ time steps. It is possible to repeat the procedure for different values of τ, so to take into consideration the nearest proximity of the node (low τ) or the broader topological role of a node (high τ, which means you let the heat flow for more time).

Then from the n-size vector obtained, the characteristic function is computed through the Fourier Transform. It is applied with different frequencies, $s_1, s_2, ..., s_S$, to grasp different information from the n-size vectors. The s values are called "`time_points`" in our code.

For this reason, the output embedding dimension is fixed by the following formula: $d(EMB) = 2 \cdot S \cdot T$, where T is the number of taus and S is the number of frequencies used by Fourier Transform.

4.5 Deep embeddings

We used also some deep embedding methods, using Graph Neural Networks, with a test of the GATher method and the graphSAGE algorithm (which makes use of a Graph Convolutional Network).

The input vector of the network for each node is a vector with only one feature, the node degree.

4.5.1 graphSAGE

It is a GCN where the AGGREGATE function is generalized (not equal to the sum of the neighbours, but in this case is the mean of the neighbours of the node). Moreover, the input to the UPDATE function is not the sum of the weighted transformation of the embedding of the node itself and the weighted transformation of the aggregation of the neighbours, but it is the concatenation of the two. This design rule adds more input information to the net (and also more parameters to train, but just for the input layer of the net). The implementation relies on the use of the SAGEConv class from `torch_geometric.nn` library.

The `_prepare_data` method serves as a bridge between the networkx graph to a PyTorch Geometric Data object. In addition the node degree feature is added.

For the GraphSAGEModel we decided to use a net with 3 layers, the input one, a hidden one and the output layer, which would return the final embedding of the node.

The width of the hidden channel has been set to 64, while the final embedding dimension to 16.

4.5.2 GATher

The same has been made with GATher method, with the same hyperparameters. The only difference is that it has been used a GATModel instead of a GraphSAGEModel in the ml model definition and the class `conv.GATConv` as layer of the net.

The GATher idea is another type of AGGREGATE implementation. Instead of a mean or normalized mean of the embeddings of the neighbours, the message passed to the update function is a weighted sum of the neighbours of the node, where each aggregation weight is obtained applying a type of softmax to a value e. This value is the dot product between the attention vector a and the concatenation of the preprocessed embeddings of the 2 vectors for which we want to calculate the aggregation weight. Preprocessed is the application of a dense layer to the embeddings h_v and h_u of the nodes.

4.5.3 Basic GCN

We also implemented a deep embedding through the use of GCNConv class for the layers of the GCN.

Here the aggregate function to create the message $m_{N(u)}^{(k)}$ is not the mean of the neighbours of u, but the mean normalized by the degree of the two nodes. Moreover, self loops are used.

5 Machine Learning

5.0.1 Feature Engineering

Edge features are created by `create_feature_vectors()` in `evaluation.py`, which combines node embeddings using binary operators defined in `config.py`: Hadamard: $u * v$ (element-wise product), Average: $(u + v)/2$, L1: $|u - v|$ (absolute difference), L2: $(u - v)^2$ (squared difference)

5.0.2 Classification

`run_single_fold()` in `run_pipeline.py` implements the training process: first it constructs feature matrices separately for positive and negative samples to ensure both classes are represented. Then combines and shuffles training data. Trains a logistic regression classifier (scikit-learn). Then generates probability predictions for test edges

5.0.3 Evaluation

The `evaluate_predictions()` function in evaluation.py computes multiple metrics: AUC-ROC is the primary metric measuring classification performance across all thresholds. Accuracy, Precision, Recall, F1-Score are the secondary metrics at 0.5 threshold. Results are aggregated across folds using `aggregate_cv_results()`, reporting mean ± standard deviation for each metric.

6 Results

We report the results (Table 2) of the runs of the various embedding methods over the final heterogeneous dataset. There is the runtime of the only run performed, the AUC-ROC metric obtained by it along with the size of the embedding and the peak RAM used during the calculation.

Node2Vec and GraphWave results are omitted due to computational timeouts on the DEI server.

Model	Time (s)	ROC-AUC	Embedding Size (bit)	Peak Memory (GB)
DeepWalk	5 527	0.9120	32	33.4
Node2Vec	–	–	–	–
MetaPath2Vec	22 981	0.9791	32	25.7
GraphWave	–	–	–	–
GATher	35 230	0.9932	16	156.5
GraphSAGE	12 194	0.9945	16	27.9
GCN	17 278	0.9893	16	30.6

Table 2: Performance comparison of node embeddings methods

The deep embedding methods (GraphSAGE, GATher, and GCN) consistently outperformed shallow methods, with GraphSAGE achieving the highest ROC-AUC (0.9945). This suggests that the message-passing architecture is highly effective at capturing the complex biological relationships in a Gene-Disease-Drug network.
Among the shallow methods, MetaPath2Vec (0.9791) significantly outperformed DeepWalk (0.9120). This confirms

that in a heterogeneous graph, guiding random walks with specific semantics (e.g., Gene-Disease-Gene) captures much more relevant biological context than unbiased random walks.

GraphSAGE offers the best balance. It achieved the highest ROC-AUC while being the fastest deep method (12 194 s) and maintaining a relatively low memory footprint (27.9 GB). Its "mean aggregation" and concatenation strategy seem particularly robust for this specific heterogeneous dataset.
DeepWalk remains the fastest overall (5 527 s), making it a viable baseline for quick iterations, though at the cost of $\sim 8\%$ lower accuracy compared to deep methods.

7 Conclusions

This study evaluated the performance of various node embedding techniques for link prediction within a large-scale, heterogeneous biological network. Our results demonstrate that while shallow embedding methods like DeepWalk and metapath2vec provide a computationally efficient baseline, deep learning architectures offer superior predictive accuracy.

The experiments highlight three key insights:

1. GraphSAGE emerged as the most balanced model, providing the highest accuracy with significantly lower memory requirements and runtime compared to attention-based models like GATher.

2. The success of metapath2vec over DeepWalk underscores the importance of incorporating node-type semantics when dealing with disease-gene-drug interactions.

3. Deep methods proved highly effective even with low-dimensional (16-bit) embeddings, suggesting that non-linear aggregation functions capture biological topologies more densely than random-walk-based skip-gram models.

8 Future work

8.1 Dataset improvements

The current study lays the foundation for further refinements in data collection and processing. Several avenues for improvement have been identified.

- *Integration of Premium Data Sources:* The present dataset is based exclusively on open-access data. Future iterations could benefit from the inclusion of proprietary or paid datasets, which may provide more granular information and a more comprehensive view of the phenomenon under investigation.

- *Expert-Led Feature Selection:* During the dataset construction phase, certain decisions were made using randomized approaches. A more robust methodology would involve consulting domain experts to guide the selection process, ensuring that the final dataset reflects more accurate and contextually relevant criteria.

- *Optimization of Data Processing Pipeline:* Technical constraints encountered during the local storage of data structures in RStudio limited the final sample size used in our experiments. While the dataset analysed in Section 2 is representative, it is a subset of a much larger potential graph. Specifically, our data pipeline is capable of generating a network comprising 106 136 nodes and 57 768 268 edges. Future work will focus on optimizing memory management and computational workflows to utilize this full-scale graph.

8.2 What has not been implemented from our plan

We have not made graphwave sensitive to different types of node. We have not been able to test node2vec with the total dataset, and so we have not found the optimal p and q values for the biased random walks. The metapath random walks moreover have not been tried with a "node2vec-type" random walk. We have not been able to test node2vec and graphwave on the final dataset for time restriction reasons (we have not found a way to make the biased random walk generation and the wavelet calculation fast).

For the deep embeddings, just a feature was used (the node degree), but this could bring the net to oversmooth and capture only a side of the node importance. It could then be poossible to add some centrality measures (not done due to runtime constraint) or the type of the node (as a one-hot vector in $\mathbb{R}^3$ for genes-disease-drug).

8.3 Open questions

The consistently high AUC-ROC values across all models (>0.90) warrant further investigation into potential structural biases within the dataset, such as node degree correlation. Furthermore, while 16-bit and 32-bit embeddings provided excellent results, a sensitivity analysis of the embedding dimensionality is required to identify the optimal trade-off between predictive power and computational overhead.

Members contributions

40% Lorenzo: Implementations of metapath2vec, graphwave, graphSAGE, GATher. Tests on local pc and on DEI server.

25% Minseok: Implementation of core modules, utils and pipeline. Base implementation of embeddings.

35% Matteo: Creation of the final dataset.

Use of AI

- Used Gemini to write and improve the R code to create the final dataset
- Code written entirely with the help of ChatGPT or Visual Studio Agent
- Troubleshooting for optimization issues (for RAM optimization during local tests, for time optimization during tests on DEI server)
- Utilized different LLMs for proofreading and improving the English phrasing of this report

References

[1] Lorenzo Berlese, Minseok Kim, and Matteo Meneghin. Learningfromnetwork-project. `https://github.com/Lorenxo-Padova/LearningFromNetwork-project`, 2024. GitHub repository.

[2] Claire Donnat, Marinka Zitnik, David Hallac, and Jure Leskovec. Spectral graph wavelets for structural role similarity in networks. *CoRR*, abs/1710.10321, 2017.

[3] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. *CoRR*, abs/1607.00653, 2016.

[4] Francesco Madeddu, Lucia Testa, Gianluca De Carlo, Michele Pieroni, Andrea Mastropietro, Aris Anagnostopoulos, Paolo Tieri, and Sergio Barbarossa. Vitagraph: Building a knowledge graph for biologically relevant learning tasks, 2025.

[5] Matteo Meneghin. Data processing. `https://github.com/Lorenxo-Padova/LearningFromNetwork-project/blob/main/data/data_processing.Rmd`, 2024. GitHub file.

[6] Matteo Meneghin. Dataset lfn project. `https://drive.google.com/drive/u/1/folders/1F3n5tjf0gsek4djYCxxCQKrrF9OEfnT6`, 2026. Drive directory.

[7] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. *CoRR*, abs/1403.6652, 2014.